

第 1 周学习报告：CS231n Assignment 1

WalkerZJH

2026.05.18 – 2026.05.24

1 任务目标

本周工作围绕 CS231n Assignment 1 展开，目标是完成深度学习与基础分类器相关代码补全，并整理为可复现、便于公开查看和后续维护的仓库材料。当前仓库将过程代码、学习笔记、实验结果和阶段性成果分开组织：

- 过程代码位于 `units/assignment1/code/`。
- 学习笔记、项目结构说明和实现规划位于 `units/assignment1/notes/`。
- 实验结果和可视化图片位于 `units/assignment1/results/`。
- 阶段性报告归档位于 `deliverables/week01_assignment1/`。

2 从 Jupyter Notebook 到纯 Python 工程的整理思路

原始 CS231n Assignment 1 以多个 Jupyter Notebook 作为任务入口，每个 Notebook 同时承担说明、调试、实验和结果展示功能。这种形式适合课程交互式学习，但不利于后续仓库维护、自动化运行和快速索引代码。因此本次整理采用“保留核心实现、移除交互式外壳、统一实验入口”的方式，将 Assignment 1 转换为纯 Python 工程结构。

转换时遵循以下原则：

1. **Notebook 只作为任务理解来源**：读取 Notebook 中的实验目标、调用顺序和检查点，但不将 Notebook 继续作为正式运行入口。
2. **核心逻辑下沉到模块**：将 kNN、Softmax、SVM、全连接网络、基础层和优化器的 TODO 补全在 `cs231n/` 模块内，避免把重要实现散落在 Notebook 单元格中。
3. **统一实验脚本**：用 `run_assignment1_experiments.py` 串联数据加载、模型训练、评估、日志写出和图表生成，使实验可以通过一条命令复现。
4. **结果与报告分离**：实验原始输出和轻量结果放在 `units/assignment1/results/`，阶段性报告放在 `deliverables/week01_assignment1/latex/` 和 `pdf/`。

3 模块架构思路

Assignment 1 当前代码架构按功能拆分为四层：

- **数据层**: `cs231n/data_utils.py` 负责读取 CIFAR-10 数据, 并提供训练集、验证集和测试集划分。
- **基础算子层**: `cs231n/layers.py` 实现 affine、ReLU、归一化、dropout、卷积、池化和损失函数等可复用组件。
- **模型层**: `cs231n/classifiers/` 中实现 kNN、线性分类器、Softmax、SVM、TwoLayerNet 和 FullyConnectedNet。
- **训练与实验层**: `cs231n/solver.py`、`cs231n/optim.py` 和 `run_assignment1_experiments.py` 负责参数更新、训练循环、评估和结果保存。

这种拆分使模型实现可以复用底层 layer, 训练脚本只负责组织流程, 不直接堆叠数学细节。后续如果继续推进 Task1 NumPy 神经网络框架, 也可以沿用“基础算子、模型、优化器、实验入口”这一结构。

4 主要模块实现思路与易错点

4.1 kNN

kNN 的核心在于距离矩阵计算和多数投票预测。实现中补全了两层循环、一层循环和无循环三种 L2 距离计算方式, 其中无循环版本使用公式

$$\|x - y\|_2^2 = \|x\|_2^2 + \|y\|_2^2 - 2x^\top y$$

避免显式遍历样本对。预测阶段对每个测试样本选取距离最近的 k 个训练样本标签, 并用多数投票得到类别。

易错点主要有三处: 第一, 广播维度必须保持为 `(num_test, num_train)`; 第二, 平方距离由于浮点误差可能出现极小负数, 开方前需要用 `maximum` 做保护; 第三, 投票平票时应选择较小标签, 保持结果稳定。

4.2 Softmax 与 SVM

Softmax 实现包括朴素版本和向量化版本。关键步骤是先对 score 减去每个样本的最大值, 避免指数溢出, 再计算交叉熵损失和权重梯度。SVM 实现使用 multiclass hinge loss, 重点是构造 margin 矩阵, 并对正确类别补回每行梯度的负计数。

易错点包括: 正则项损失通常写为 $reg \sum W^2$, 梯度为 $2regW$; Softmax 的概率矩阵需要先复制或原地调整正确类别项; SVM 中正确类别的 margin 必须清零, 否则会错误增加损失。

4.3 LinearClassifier

`LinearClassifier` 是 Softmax 和 SVM 的公共训练框架, 负责小批量采样、调用具体 loss、执行 SGD 更新和预测。该层的设计重点是“分类器损失函数”和“训练流程”解耦, 子类只需要实现 `loss()`, 训练逻辑可以复用。

易错点是 mini-batch 采样要同时抽取 X 和 y , 并保持索引一致; 预测时应对 $X \cdot W$ 取最大 score 的类别, 而不是对概率或损失进行额外处理。

4.4 基础层 layers.py

`layers.py` 是全连接网络的底层基础, 包含 affine、ReLU、batch normalization、layer normalization、dropout、卷积、池化、spatial normalization 以及 SVM/Softmax loss。实现思路是每个 forward 函数返回输出和 cache, 每个 backward 函数只依赖对应 cache 还原局部梯度。

关键点是严格维护张量形状: affine 层需要把输入展平为 (N, D) ; 卷积和池化层需要处理 padding、stride 和输出尺寸; batch normalization 的训练模式和测试模式行为不同; dropout 在训练阶段随机屏蔽, 在测试阶段直接返回输入。

易错点主要是 cache 内容不完整会导致反向传播无法恢复中间变量; batch normalization 的 running mean 和 running variance 需要在训练时更新; layer normalization 与 batch normalization 的归一化维度不同, 不能直接照搬 axis。

4.5 TwoLayerNet 与 FullyConnectedNet

TwoLayerNet 使用 affine-ReLU-affine-softmax 结构, 适合作为从线性分类器过渡到神经网络的最小例子。FullyConnectedNet 则将该结构推广到任意隐藏层数量, 并可选支持 batch normalization、layer normalization 和 dropout。

实现上, 前向传播按层缓存中间结果, 反向传播按相反顺序依次计算梯度。每个权重矩阵都加入 L2 正则化, 损失中加入 $0.5\text{reg} \sum W^2$, 这样梯度项可以简洁写为 $\text{reg}W$ 。

易错点是层编号和参数名必须一致, 例如 `W1, b1, W2, b2`; 使用 batch normalization 时还要维护 `gamma` 和 `beta`; dropout cache 必须按层保存, 否则反向传播会复用错误 mask。

4.6 优化器 optim.py

优化器模块实现了 SGD、SGD with momentum、RMSProp 和 Adam。SGD 是基础更新; momentum 通过速度项平滑梯度; RMSProp 根据梯度平方的滑动平均调整学习率; Adam 同时维护一阶矩和二阶矩, 并做 bias correction。

易错点是优化器的 `config` 需要在多次调用间保留状态; Adam 的时间步 `t` 必须递增; RMSProp 和 Adam 中的 `epsilon` 应加在分母, 防止除零。

5 数据与实验设置

数据集按 `cs231n/datasets/get_datasets.sh` 的要求下载。

实验脚本为 `units/assignment1/code/run_assignment1_experiments.py`。本次实验使用 CIFAR-10 子集:

- 训练集: 2000 张图片。
- 验证集: 500 张图片。
- 测试集: 500 张图片。
- kNN 使用当前实验子集的完整 2000 张训练样本、500 张验证样本和 500 张测试样本, 不再额外采样。

6 实验结果

模型	设置	训练准确率	验证准确率	测试准确率
kNN	$k = 7$, train=2000, val=500, test=500	—	0.2800	0.2320
Softmax	lr= 10^{-7} , reg= 2.5×10^4 , iters=500	0.3410	0.3080	0.2960
TwoLayerNet	sgd_momentum, lr=0.001, epochs=3	0.2560	0.2820	0.2760
FullyConnectedNet	Adam, lr=0.001, epochs=3	0.5700	0.2860	0.2320

表 1: Assignment 1 小规模 CIFAR-10 实验结果

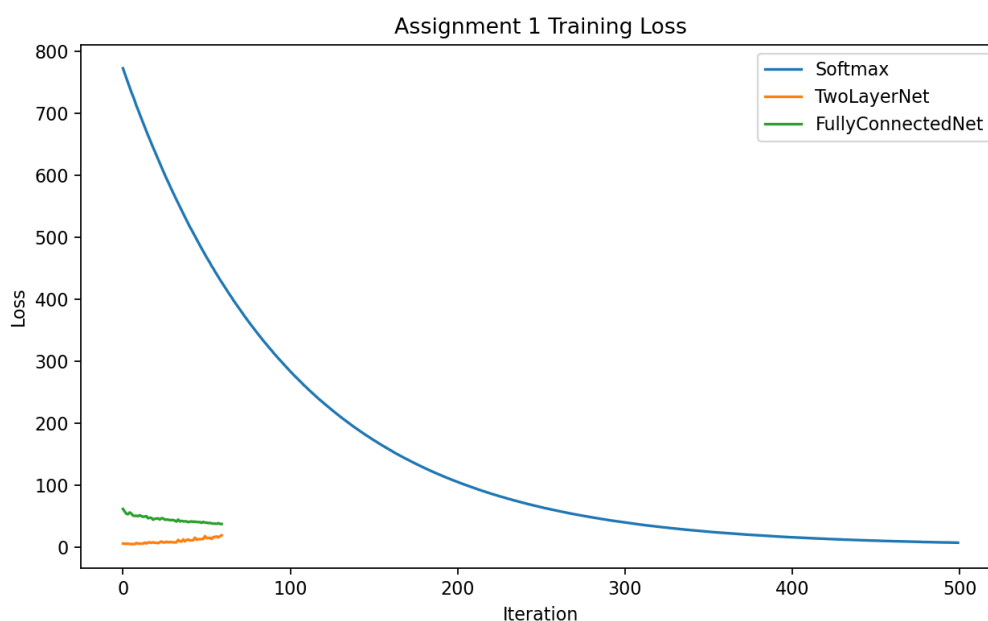


图 1: Softmax、TwoLayerNet、FullyConnectedNet 训练损失曲线

7 结果分析

从实验结果看, kNN 使用 2000/500/500 实验子集后, 最佳验证准确率为 0.2800, 测试准确率为 0.2320。kNN 不需要训练参数, 但推理阶段需要计算测试样本到所有训练样本的距离, 因此样本数增加后运行时间会明显上升。

Softmax 在当前小规模设置下取得最高测试准确率 0.2960, 说明线性分类器已经能从原始像素中学习一部分 CIFAR-10 判别信息。TwoLayerNet 的测试准确率为 0.2760, 接近 Softmax, 但训练轮数较少, 后续仍需要调节学习率、隐藏层规模和正则化强度。

FullyConnectedNet 的训练准确率为 0.5700, 高于验证和测试准确率, 说明模型容量变大后已经出现过拟合。后续可以通过扩大训练集、增加正则化、调整 dropout keep ratio、延长训练并配合学习率衰减来改进泛化能力。

8 本周总结

本周完成了 Assignment 1 的主要代码补全、数据下载、实验脚本、实验结果和阶段性报告整理。当前仓库已经清理原始 notebook、notebook 转换产物和当前实验不再使用的辅助脚本，只保留复现实验所需的核心代码、学习笔记、实验结果和阶段性归档材料。通过本次整理，Assignment 1 已经从课程 Notebook 形态转为更清晰的 Python 模块化工程，便于公开查看和后续继续推进 Task1 NumPy 神经网络框架。